



Port City International University

Excellence in Higher Education

UGC & Govt. Approved University in Khulshi in Chattogram

Report on

Student Document Portal Approval System

Course code : CSE 322

Course Title : Software Engineering Sessional

Submitted By

Name of Student : Manik Mia (ID: CSE 03007870)

Antar Barua (ID: CSE 03007865)

Samiya Jahan ID: CSE 03007852)

Batch No : 30th (B1)

Department of Computer Science and Engineering

Submitted To

Name of Lecturer : Jahid Karim Fahim

Department : CSE

Port City International University

Date of Submission:

22/11/2025

INDEX

SL No.	Title	Page No.
01	Experiment Name	01
02	Introduction	02
03	Procedure	03–05
04	Output	06–08
05	Database Implementation	09
06	Discussion	10
07	References	11

Experiment Name: Student Document Portal Approval System

Introduction: The PCIU Student Portal is a full-featured academic management system created to digitalize student activities of Port City International University (PCIU). The purpose of this system is to provide a smooth, automated, and organized way for both students and administrators to manage academic information.

The portal has two separate access levels:

Student Panel

Students can:

- Log in securely
- View dashboard
- Check personal profile
- View enrolled courses
- View published exam results
- Read and download notices
- Update basic information
- Log out securely

Admin Panel

Admin can:

- Add new students
- Update existing student information
- Publish academic notices
- Manage student results
- Manage course list
- Control entire student database

This system improves accuracy, reduces manual workload, and creates a structured workflow for university management.

Procedure:

Step–1: Extract Project

- Open the project folder in VS Code

Step–2: Install Required Packages

Open terminal inside the project directory:

```
npm install
```

This will install all backend dependencies such as Express, JWT, bcrypt, etc.

Step–3: Add Environment Variables

Create a `.env` file and add:

```
PORT=5000
DB_HOST=localhost
DB_USER=root
DB_PASS=manikpciu
DB_NAME=pciu_student_portal
JWT_SECRET=your_secret_key
JWT_EXPIRES_IN=7d
```

Purpose:

- Secure login
- Protect database credentials

Step–4: Create MySQL Database

1. Open MySQL CLI
2. Create database:

```
CREATE DATABASE pcIU_student_portal;
```

3. Import provided SQL file to create tables:

- Users
- Documents

Step–5: Start Backend Server

Run:

```
npm start
```

If nodemon installed:

```
npm run dev
```

Expected message:

```
Server running on port 5000
Database connected successfully
```

Step–6: Run Frontend

Open:

```
index.html
```

This loads:

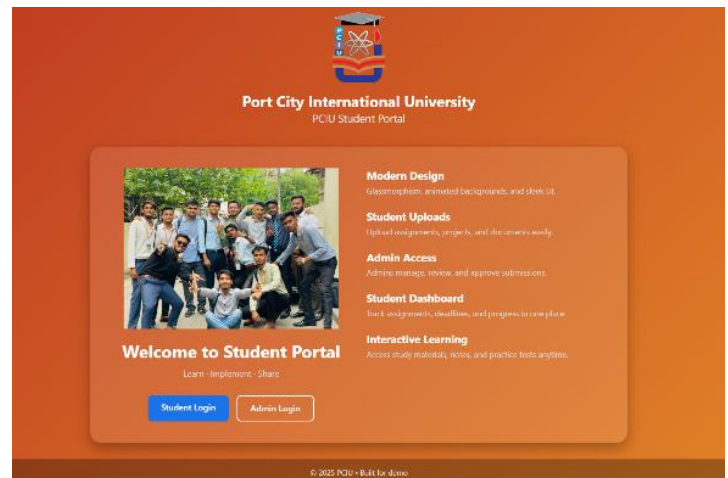
- Student Login Page
- Admin Login Page (different route)

Output:

Successful Login

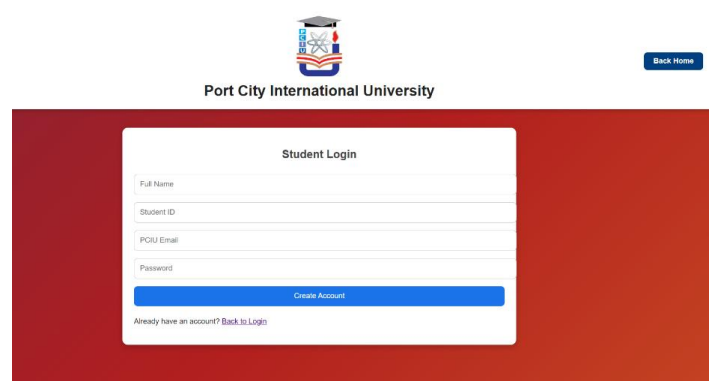
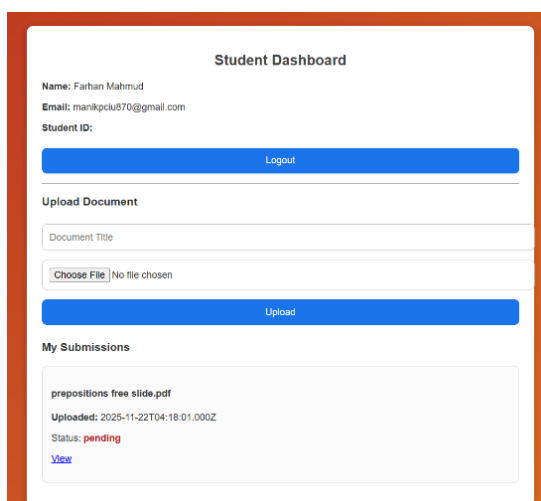
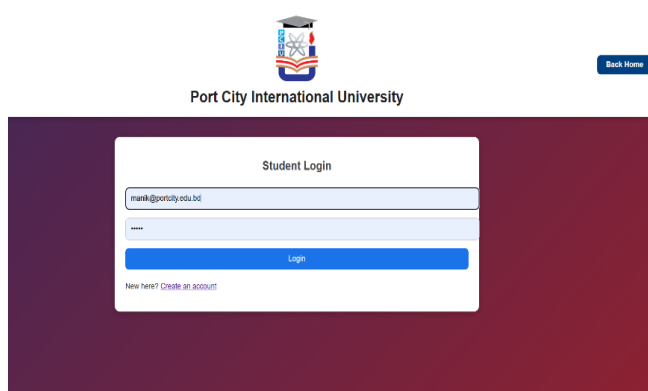
If credentials are correct:

- Student → Student Dashboard
- Admin → Admin Dashboard



Student Dashboard Output

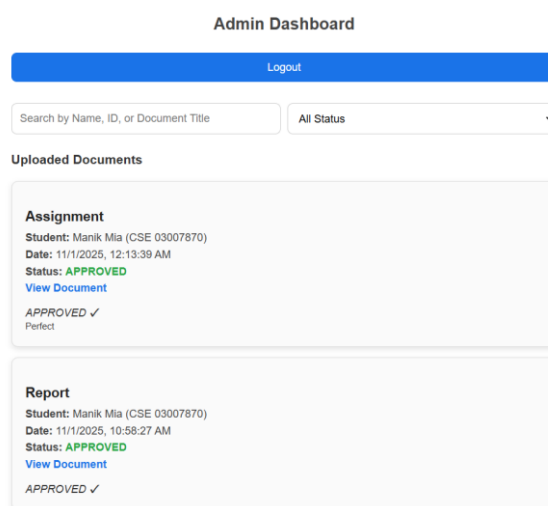
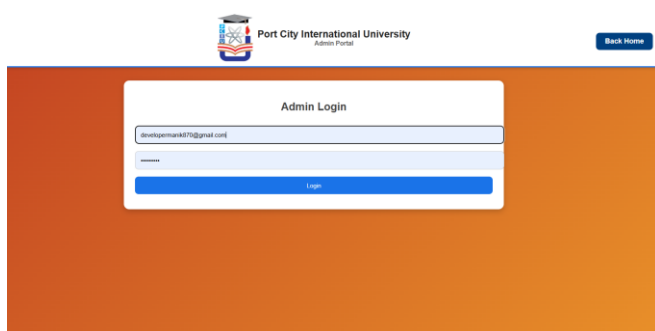
- Student name
- Student ID
- Email
- Password



Admin Dashboard Output:

For Admin Login:

- Email
- Password
- Login

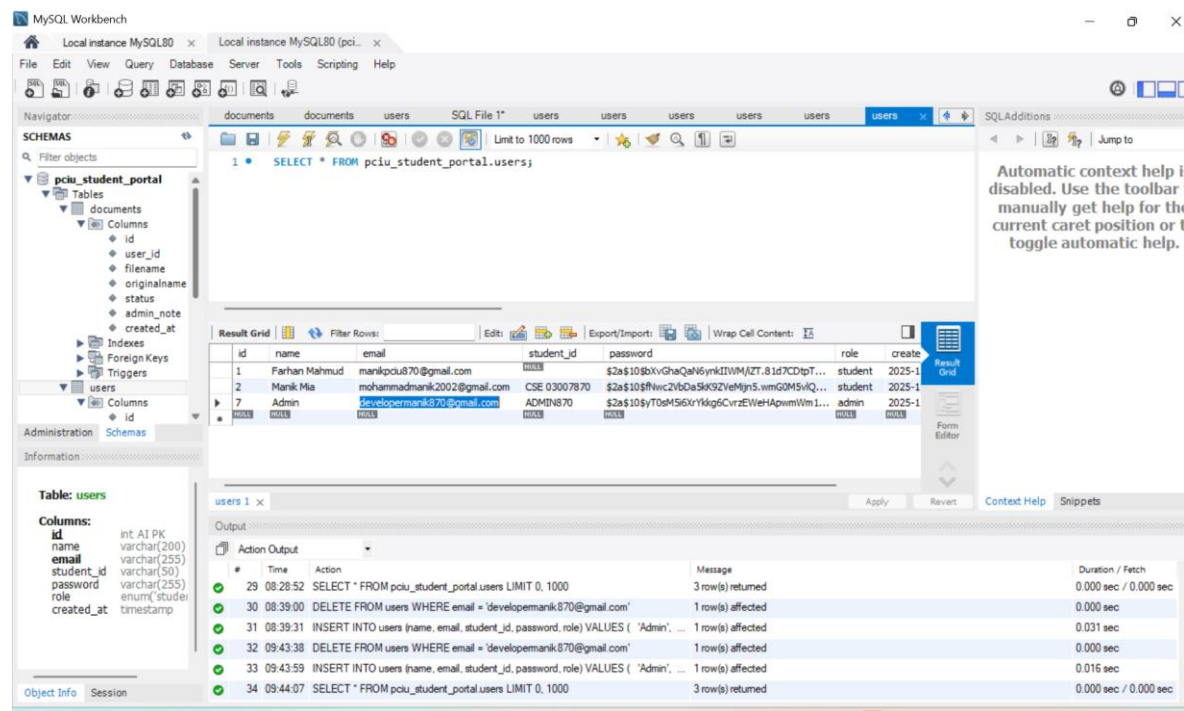


Database Implementation:

For this project, the **MySQL** relational database management system was used to store and manage all authentication and portal-related data. The database contains a table named **users**, which holds records for both students and administrators. Each user entry includes fields such as **id**, **name**, **email**, **student_id**, **password**, **role**, and **created_at**. Passwords are securely stored using **bcrypt hashing**, ensuring that no raw or plain-text passwords are saved in the system.

During implementation, multiple insert and delete operations were performed dynamically, as shown in the Workbench log. New users (including admin accounts) were added through queries such as `INSERT INTO users (...) VALUES (...)`, and unnecessary records were removed via `DELETE FROM users WHERE ...`. The result grid confirms that all data names, emails, student IDs, roles, and hashed passwords—was stored successfully and is fully synced with the backend authentication system.

This database structure ensures **secure login**, **role-based access**, and **integrated CRUD operations** throughout the PCIU Student Portal.



Discussion:

In this project, we updated the PCIU Student Portal by adding a **Document Approval System** where students can upload their required documents, and admins can approve or reject them from the dashboard. To improve communication, an **automatic email notification feature** was implemented.

Whenever an admin **approves** a document, the student instantly receives a confirmation email. If the document is **rejected**, the system sends an email informing the student along with the reason.

This update makes the portal more professional, increases transparency, and reduces delays by keeping students informed without manually checking the system. The addition of real-time notifications significantly improves both the administrative workflow and the overall user experience.

References:

1. Express.js Documentation — *Backend routing & server creation*
<https://expressjs.com/>
2. Node.js Official Documentation — *Runtime environment used for backend development*
<https://nodejs.org/en/docs>

3. MySQL Documentation — *Database design, query management & table structure creation*
<https://dev.mysql.com/doc/>
4. JSON Web Token (JWT) — *Student and admin login authentication*
<https://jwt.io/>
5. Nodemailer Documentation — *Email sending system for document approval/rejection notifications*
<https://nodemailer.com/about/>
6. Bcrypt.js Documentation — *Password hashing for student & admin accounts*
<https://github.com/dcodeIO/bcrypt.js>
7. W3Schools HTML, CSS, JavaScript References — *Frontend UI development & form handling*
<https://www.w3schools.com/>